

## Annexe G

# TD5 : Construction d'une matrice d'Oracle

### G.1 Travail demandé

#### G.1.1 Construction d'un oracle générique

1. En utilisant le package python `numpy`, construire une fonction qui construit la matrice d'un oracle associé à une fonction booléenne quelconque  $f$ , de  $n$  bits vers  $m$  bit. Ses arguments sont
  - le nombre  $n$  de bits d'entrée (les qubits de data)
  - le nombre  $m$  de bits de sorties (les qubits ancillaires)
  - la fonction booléenne  $f$ , codée en python

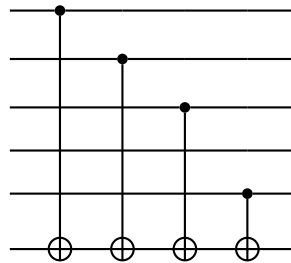
Le prototype de cette fonction est donc du type

```
def build_matrix(n: int, m : int, f) -> np.ndarray :  
    """ Build an oracle associated with a bool function  
  
    Params:  
        n and m are integers,  
        f is a bool functions from n bits to m bits  
    Returns:  
        a unitary matrix that represent the quantum Oracle  
        associated with f (n qubits of data and m ancillary qubits)  
    """
```

La matrice produite ici est réelle, donc son adjointe est sa transposée. Vous écrirez le code qui permet de vérifier que la matrice oracle que vous avez produit est bien unitaire en la multipliant par sa transposée et en vérifiant que le résultat est bien l'identité.

### G.1.2 Algorithme de Bernstein-Vazirini

On considère le circuit à 6 qubits suivant



Ce circuit sera notre oracle dans le cadre d'un algorithme de Bernstein-Vazirini sur 6 qubits.

Cet oracle utilise ses 5 premiers qubits pour la donnée d'entrée, le sixième est un qubit ancillaire. Cet oracle dissimule une fonction "produit pointé" entre une valeur secrète et la valeur d'entrée.

Le sixième et dernier qubit est le qubit ancillaire (c'est une fonction booléenne de 5 bits vers 1 seul bit).

#### Travail demandé

1. Implémenter l'algorithme de Bernstein-Vazirini avec cet oracle, en utilisant myQLM.
2. Quelle est la valeur du secret caché dans cet oracle ?
3. En examinant la structure du circuit de l'oracle, en déduire une manière d'encoder un produit pointé avec n'importe quelle valeur secrète.
4. donner un exemple en construisant un nouvel oracle pour une valeur que vous choisirez, et l'illustrer en faisant tourner votre programme myQLM pour retrouver cette valeur.

## G.2 Solution

### G.2.1 Construction d'un oracle générique

Voici le module `BV_Compute_Matrix.py` permettant de calculer une matrice d'oracle pour une fonction booléenne générique

```
import numpy as np
import math
import sys

# int to binarray
def int_to_binarray(a: int, n: int) -> list:
    """ Convert an integer to a binary array (or binarray)

    Params:
        a is an integer
        n is the number of bits encoding a
    Returns:
        the binarray representing a
    """
    fmt = "0%db" % (n)
    return [int(bit) for bit in format(a, fmt)]

def binarray_to_int(bina: list) -> int :
    """ Convert binarray to integer

    Params:
        bina is a binaryarray
    Returns:
        the integer in base 10
    """
    n = len(bina)
    r = 0
    for i in range(0, n):
        r = r + np.power(2,i)*bina[n-1-i]
    print(bina)
    print("r=%d" % (r))
    return r
```

```

# binary dot product
def bitwise_dot_product(a: int , b: int, n: int) -> int:
    """ Compute the bitwise dot product between two integers

    Params:
        a is an integer
        b is an integer
        n is the number of bits encoding a and b
    Returns:
        the dot product as an integer
    """
    fmt = "0%db" % (n)

    bit_array_a = int_to_binarray(a,n)
    bit_array_b = int_to_binarray(b,n)

    s=0
    for i in range(0,n):
        s = s + (bit_array_a[i] ^ bit_array_b[i])

    sm2 = s %2
    print("a=%d b=%d n=%d s=%d sm2=%d" % (a, b, n, s, sm2))
    return sm2

def bool_func(x: int, n : int, m: int ) -> int:
    """ A dummy bool function for building the oracle

    Params:
        x is an integer
        n is the number of bits for the input n
        m is the number of bits for the output m
    Returns:
        always returns 1
    """
    return 1

```

```

# k is an integer  $0 \leq k \leq 2^{(n-1)}$ 
# n is the number of qubits
def int_to_bra(k : int, n: int) -> np.ndarray :
    """ Convert an integer k to the k-th basis vector as a bra

        Params:
            k is an integer
        Returns:
            the related bra
    """
    out = [0]*np.power(2,n)
    out[k] = 1
    return np.array([out])

def int_to_ket(k: int, n: int) -> np.ndarray :
    """ Convert an integer k to the k-th basis vector as a ket

        Params:
            k is an integer
        Returns:
            the related ket
    """
    return np.transpose(int_to_bra(k,n))

def binarray_to_bra( bina: list ) -> np.ndarray :
    """ Convert binarray to bra
        Params:
            bina is an binary array
        Returns:
            the bra related to the k-th basic vector
            as k is the integer value associated with bina
    """
    return int_to_bra(binarray_to_int(bina), len(bina))

def binarray_to_ket( bina: list ) -> np.ndarray :
    """ Convert binarray to ket

```

```

        Params:
            bina is an binary array
        Returns:
            the ket related to the k-th basic vector
            as k is the integer value associated with bina
    """
    return int_to_ket(binarray_to_int(bina), len(bina))

# n qubits en input
# m qubits ancillaires
def build_matrix(n: int, m : int, f) -> np.ndarray :
    """ Build an oracle associated with a bool function

        Params:
            n and m are integers,
            f is a bool functions from n bits to m bits
        Returns:
            a unitary matric that represent the quantum Oracle
            associated with f (n qubits of data and m ancillary qubits)
    """
    init = True
    fmt = "0%db" % (n+m)
    for v in range(0, np.power(2,n+m)):
        # Compute binarray for v
        v_array = int_to_binarray(v, n+m)

        # |x> is the first n qubits of |v>
        x = v_array[0:n]
        xb = binarray_to_int(x)

        # |x> is the last m qubits of |v>
        y = v_array[n:n+m]
        yb = binarray_to_int(y)

        # Compute f(x)
        fx = f(xb, n, m)

```

```

#Compte y XOR f(x)
yfx = yb ^ fx

print("v=%s" % (format(v, '03b')))
print("v=%s x=%s y=%s f(x)=%s yfx=%s" %
      (format(v, fmt), format(xb, fmt),
       format(yb, fmt), format(fx, fmt), format(yfx, '01b')))
# Build related result binarray
yfx_array = int_to_binarray(yfx, m)
print(yfx_array)
x_oplus_yfx_array = x + yfx_array
print(x_oplus_yfx_array)
print("----")

# Build matrix
if init:
    # This is the first iteration, matrix is the column ket
    matrix = binarray_to_ket(x_oplus_yfx_array)
    init = False
else:
    # hstack the new ket to the previous ones
    matrix = np.hstack((matrix, binarray_to_ket(x_oplus_yfx_array)))

print(matrix)
return matrix

```

Il s'utilise de la manière suivante

```

#!/usr/bin/env python
import numpy as np
import math
import sys

import BV_Compute_Matrix as bvm

# Set inputs
n=2
m=1

```

```

SECRET=3

# Secret function
def secret_func(x: int, n : int, m: int ):
    """ The secret function for building the oracle

    Params:
        x is an integer
        n is the number of bits for the input n
        m is the number of bits for the output m
    Returns:
        the computed secret value as an integer
    """
    return bvm.bitwise_dot_product(x, SECRET, n)

# Main
# Avoid truncation when printing numpy matrices
np.set_printoptions(threshold=sys.maxsize)

matrix = bvm.build_matrix(n, m, secret_func)

# Test : the matrix should be unitary. Since it's real
# we should only check if its transpose matrix is its reverse matrix
res = np.matmul(matrix, np.transpose(matrix))

# Let's print to eye-check the result
print(matrix)
print("=====")
##print(res)

# Multiply res and its transpose and check if we get the identity
if (res == np.identity(np.power(2, n+m))).all():
    print("C'est bien l'identité !!")
else:
    print("Il y a une erreur !!")

```

## G.2.2 Algorithme de Bernstein-Vazirini

Voici le code myQLM du circuit qui implémente le circuit complet incluant l'oracle

```
#!/usr/bin/env python
from qat.lang.AQASM import *
from qat.qpus import PyLinalg
import matplotlib.pyplot as plt
import numpy as np

prog = Program()
qbits = prog.qalloc(6)

# 5 qubit de données, 1 qubit ancillaire

# Porte X sur le qubit ancillaire
prog.apply( X, qbits[5])

# Bordée de porte H
prog.apply( H, qbits[0])
prog.apply( H, qbits[1])
prog.apply( H, qbits[2])
prog.apply( H, qbits[3])
prog.apply( H, qbits[4])
prog.apply( H, qbits[5])

# Injecter l'oracle de l'énoncé
prog.apply(CNOT, qbits[0], qbits[5])
prog.apply(CNOT, qbits[1], qbits[5])
prog.apply(CNOT, qbits[2], qbits[5])
prog.apply(CNOT, qbits[4], qbits[5])

# Bordée de porte H
prog.apply( H, qbits[0])
prog.apply( H, qbits[1])
prog.apply( H, qbits[2])
prog.apply( H, qbits[3])
prog.apply( H, qbits[4])
```

```

prog.apply( H, qbits[5])

circuit = prog.to_circ()
circuit.display()

job = circuit.to_job()

linalggpu = PyLinalg()

result = linalggpu.submit(job)
l = len(result);
states = ['']*l
probabilities= [0]*l

i=0
for sample in result:
    print("State",sample.state,"with amplitude",
          sample.amplitude,"and probability",
          round(sample.probability*100,2),"%")
    states[i] = str(sample.state)
    probabilities[i] = round(sample.probability*100,2)
    i = i+1

plt.bar(states, probabilities, color='skyblue')
plt.xlabel('States')
plt.ylabel('Probabilities')
plt.title('BW-CNOT')
plt.show()

```

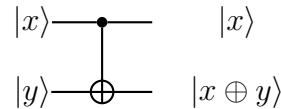
En lançant ce code on obtient le display

```
State |111011> with amplitude (0.9999999999999999+0j) and probability 100.0 %
```

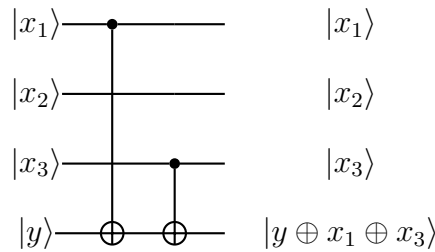
C'est donc l'état  $|11101\rangle$  qui est observé. On rappelle que, dans Bernstein-Vazirani, les qubit de données portent la valeur cible et le qubit ancillaire porte la valeur  $|-\rangle$ . J'ai positionné une porte H sur ce qubit pour le ramener à la valeur  $|1\rangle$  plus lisible (cela évite une superposition).

La valeur secrète est sur les 5 qubits de data, c'est donc 11101 soit 29 en décimal. Les bits à 1 correspondent au bits des portes CNOT dans l'oracle.

L'explication est simple. Considérons une porte CNOT. Si  $x$  et  $y$  sont des bits, sont effet est le suivant



Si j'applique



Considérons considérons l'expression  $y \oplus x_1 \oplus x_3$ , on peut l'écrire  $y \oplus f(x)$  avec  $f(x) = x_1 \oplus x_3$  si  $x = \overline{x_1 x_2 x_3}$ .

On rappelle que le produit pointé est défini par

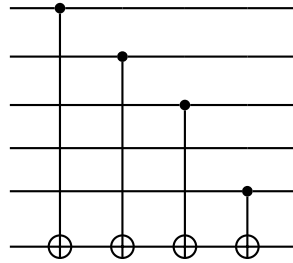
$$x \cdot y = x_0 x_0 + x_1 y_1 + \cdots + x_n y_n [2] \equiv x_0 y_0 \oplus x_1 y_1 \oplus \cdots \oplus x_n y_n$$

et donc

$$f(x) = x_1 \oplus x_3 = x_1.1 \oplus x_2.0 \oplus x_3.1 = x.101$$

On peut donc facilement encoder le produit pointé avec un secret  $S$  en positionnant des CNOT entre les qubits de data qui correspondent aux bits qui valent 1 dans  $S$  et le qubit ancillaire.

Ainsi le circuit suivant porte des contrôle sur les bits 1,2,3,5, donc le secret est 1101 qui vaut 29 en décimal.



A partir de cela, il est simple d'encoder n'importe quelle valeur en décomposant le secret en binaire.

### G.2.3 Code de l'énoncé sur 6 qubits

```
#!/usr/bin/env python
from qat.lang.AQASM import *
from qat.qpus import PyLinalg
import matplotlib.pyplot as plt
import numpy as np

n = 5

prog = Program()
qbits = prog.qalloc(n+1)

# 5 qubit de données, 1 qubit ancillaire

# Porte X sur le qubit ancillaire
prog.apply( X, qbits[n])

# Bordée de porte H
for i in range(0,n+1):
    prog.apply( H, qbits[i])

# Injecter l'oracle de l'énoncé
prog.apply(CNOT, qbits[0], qbits[5])
prog.apply(CNOT, qbits[1], qbits[5])
prog.apply(CNOT, qbits[2], qbits[5])
prog.apply(CNOT, qbits[4], qbits[5])
```

```

# Bordée de porte H
for i in range(0,n+1):
    prog.apply( H, qbits[i])

circuit = prog.to_circ()
circuit.display()

job = circuit.to_job()

linalgqpu = PyLinalg()

result = linalgqpu.submit(job)
l = len(result);
states = ['']*l
probabilities= [0]*l

i=0
for sample in result:
    print("State",sample.state,"with amplitude",
          sample.amplitude,"and probability",
          round(sample.probability*100,2),"%")
    states[i] = str(sample.state)
    probabilities[i] = round(sample.probability*100,2)
    i = i+1

plt.bar(states, probabilities, color='skyblue')
plt.xlabel('States')
plt.ylabel('Probabilities')
plt.title('BW-CNOT')
plt.show()

```

## G.2.4 Valeur générique pour un nombre quelconque de qubits et un secret variable

```
#!/usr/bin/env python
from qat.lang.AQASM import *
from qat.qpus import PyLinalg
import matplotlib.pyplot as plt
import numpy as np

# Paramètres
n = 10
secret = 325 # = (0101000101)2

# Code
prog = Program()
qbits = prog.qalloc(n+1)

# 5 qubit de données, 1 qubit ancillaire

# Porte X sur le qubit ancillaire
prog.apply( X, qbits[n])

# Bordée de porte H
for i in range(0,n+1):
    prog.apply( H, qbits[i])

# Injecter l'oracle défini par la variable 'secret'
fmt = "0%db" % (n)
secret_binary = [int(bit) for bit in format(secret, fmt)]
for i in range(0,n):
    print("i=%d bit=%d" % (i, secret_binary[i]))
    if secret_binary[i] == 1:
        prog.apply(CNOT, qbits[i], qbits[n])

# Bordée de porte H
for i in range(0,n+1):
    prog.apply( H, qbits[i])
```

```

circuit = prog.to_circ()
circuit.display()

job = circuit.to_job()

linalgqpu = PyLinalg()

result = linalgqpu.submit(job)
l = len(result);
states = ['']*l
probabilities= [0]*l

i=0
for sample in result:
    print("State",sample.state,"with amplitude",
          sample.amplitude,"and probability",
          round(sample.probability*100,2),"%")
    states[i] = str(sample.state)
    probabilities[i] = round(sample.probability*100,2)
    i = i+1

plt.bar(states, probabilities, color='skyblue')
plt.xlabel('States')
plt.ylabel('Probabilities')
plt.title('BW-CNOT')
plt.show()

```